

React TypeScript — Sprint Fullstack

L3 / B3 — React, TypeScript, Tests

Thomas Diaconu

ESIEE-IT

Programme ingénieur — ING2



Comment on travaille cette semaine

Moment	Contenu
Ce matin	2-3h de cours, puis kickoff projet
Reste du sprint	Projet, en autonomie
Fin	Review + éval individuelle

Notation

QCM (individuel) — **1/3** de la note finale · **Projet** — **2/3** de la note finale

Détail projet (sur 20) : Stories **6** · Qualité & typage **5** · Tests **3** · Git **3** · Présentation **3**

À retenir

La complétion ne suffit pas. Une story sans test ni gestion d'erreur ne compte pas.

GenAI

Autorisé et encouragé. Vous êtes responsables de **100 %** du code rendu.

Conditions :

- vous comprenez le code
- vous savez l'expliquer, ligne par ligne
- aucun plagiat

Éval individuelle

- **Explication de code** ligne par ligne
- **Revue de PR** : vous commentez un commit

Attention

Un agent écrit du code. Il ne debugge pas à votre place devant un client.

Bloc 1 — TypeScript utile

Ce qui sert vraiment en projet

Unions et narrowing

```
type Role = "USER" | "ADMIN";

function canModerate(role: Role): boolean {
  return role === "ADMIN";
}
```

Le *narrowing* : TypeScript resserre le type selon le contexte.

```
function label(v: string | null): string {
  if (v === null) return "vide";
  return v.toUpperCase(); // ici v est string, garanti
}
```

À retenir

Le narrowing permet à TypeScript de comprendre le type exact selon le contexte.

unknown vs any

```
function toErrorMessage(e: unknown): string {  
  if (e instanceof Error) return e.message;  
  return "Erreur inconnue";  
}
```

- `any` : désactive totalement TypeScript ❌
- `unknown` : force la vérification avant usage ✅

```
try { await createPost(content); }  
catch (e) { setError(toErrorMessage(e)); } // e est unknown
```

À retenir

`any` = fuite de typage. `unknown` = typage sûr.

as const et satisfies

```
const ROUTES = {  
  feed: "/feed",  
  profile: "/profile",  
} as const;  
// type: { readonly feed: "/feed"; ... } → littéraux, pas string
```

```
const config = {  
  apiUrl: "http://localhost:3000",  
  timeout: 5000,  
} satisfies AppConfig;  
// vérifié contre AppConfig, mais garde les types littéraux
```

- `as const` : fige, littéralise
- `satisfies` : vérifie **sans** élargir le type

Generics et réponses API

```
type ApiResult<T> =  
  | { ok: true; data: T }  
  | { ok: false; error: string };  
  
async function apiGet<T>(url: string): Promise<ApiResult<T>> { /* ... */ }  
  
const res = await apiGet<Post[]>("/posts");  
if (res.ok) console.log(res.data.length); // res.data typé Post[]
```

Question rapide

```
type ApiResult<T> = { ok: true; data: T } | { ok: false; error: string };  
  
const res: ApiResult<Post[]> = /* ... */;  
if (res.ok) {  
  res.data; // le type ici ?  
}
```

- A. `Post[] | undefined`
- B. `T`
- C. `Post[]`
- D. `any`

Exercice A — 10 min

À partir de :

```
type Post = {  
  id: string;  
  content: string;  
  author: { id: string; username: string };  
};
```

Exercice — 10 minutes

1. Écrire `apiGet<T>` avec le type `ApiResponse<T>` de la slide précédente
2. L'appeler pour récupérer `Post[]`
3. Gérer le cas d'erreur **sans** `any` et **sans** `as`

Bloc 2 — React, les fondations

Le socle commun, remis à plat

D'où vient React, et pourquoi

- **2013**, sorti par Facebook (Meta), open source, encore maintenu par Meta + la communauté

Le problème d'origine

Un fil d'actualité : beaucoup de données, qui changent tout le temps, à plusieurs endroits de l'écran.
À l'époque, on modifiait le DOM à la main (jQuery) :

```
$("#likes-42").text(count); // qui a mis à jour quoi, et quand ?  
$("#btn-42").addClass("active"); // l'état vrai est dans le DOM, éparpillé
```

L'idée de React

L'UI est une **fonction de l'état**. Vous décrivez à quoi l'écran doit ressembler pour un état donné.
React se charge d'aller modifier le DOM.

`UI = f(state)` — vous ne décrivez plus les transitions, vous décrivez le résultat.

Place dans le paysage frontend

	Nature	Positionnement
React	<i>bibliothèque</i> UI	vous assemblez l'écosystème vous-même
Angular	framework complet	tout fourni, opinionated, TS natif, très « entreprise »
Vue	framework progressif	entre les deux, réputé doux à apprendre
Next.js / Remix	frameworks au-dessus de React	routing, SSR, build de prod

React ne fournit **ni** routing, **ni** data fetching, **ni** gestion de formulaires. C'est pour ça que ce projet ajoute React Router, Zod, etc. Ce n'est pas un défaut, c'est le modèle.

En 2026, c'est toujours le socle le plus demandé sur le marché. D'où ce sprint.

Les particularités de React

- **Composants = fonctions.** Pas de classes (plus depuis 2019), pas de template séparé.
- **Le rendu = ré-exécuter la fonction.** Changer un état ne « patche » pas le composant : React rappelle la fonction entière et compare le résultat.
- **Markup et logique dans le même fichier**, volontairement : ce qui change ensemble vit ensemble.

(One-way data flow et Virtual DOM : voir le vocabulaire, slide suivante.)



Vocabulaire React — L'essentiel

- **Composant** : une fonction qui retourne du JSX
- **Props** : les données en **entrée**, immuables, venues du parent
- **State** : les données **internes**, qui déclenchent un re-rendu quand elles changent
- **Hook** : fonction spéciale qui branche un composant sur React (`useState` , `useEffect` ...)
- **One-way data flow** : props vers le bas, événements vers le haut
- **Virtual DOM** : le diff en mémoire avant l'écriture dans le vrai DOM

```
type ButtonProps = {  
  label: string;           // prop  
  onClick: () => void;      // événement qui remonte  
};  
  
function Button({ label, onClick }: ButtonProps) { // composant  
  return <button onClick={onClick}>{label}</button>; // JSX  
}
```

Props

Les **props** sont les paramètres d'entrée d'un composant : elles descendent du parent vers l'enfant, jamais l'inverse.

```
type UserCardProps = {
  username: string;
  isOnline: boolean;
};

function UserCard({ username, isOnline }: UserCardProps) {
  return <p>{username} {isOnline ? "●" : "●"}</p>;
}

<UserCard username="alice" isOnline={true} />
```

- **Toujours en lecture seule** : un composant ne modifie jamais ses props
- Si une valeur doit changer localement, ce n'est plus une prop : c'est un **state**, initialisé depuis la prop si besoin

JSX — ce que c'est vraiment

JSX n'est **pas** du HTML dans du JS : c'est du **sucre syntaxique**.

```
const el = <p className="bio">{user.email}</p>;
```

...est compilé en un simple appel de fonction :

```
const el = jsx("p", { className: "bio", children: user.email });
```

À retenir

Un élément JSX est donc **une valeur JavaScript** comme une autre : on peut le stocker dans une variable, le mettre dans un tableau, le retourner d'une fonction.

```
const badge = isAdmin ? <Badge /> : null; // une valeur, tout simplement
```

JSX — les règles qui vous feront perdre du temps

- **Un seul parent** par retour. Sinon, un fragment : `<>...</>`
- `className` et `htmlFor` : `class` et `for` sont des mots réservés en JS
- **Attributs en camelCase** : `onClick`, `tabIndex`, `aria-label` (les `aria-*` / `data-*` gardent leurs tirets)
- `{}` = une expression, pas une instruction : pas de `if` ni de `for` à l'intérieur
- **Balises auto-fermantes obligatoires** : `<img />`, `<br />`

```
{posts.map((post) => (  
  <PostCard key={post.id} post={post} />    // key = id métier, jamais l'index  
))}
```

Hooks — vue d'ensemble

Hook	Sert à
<code>useState</code>	déclarer un état local, qui persiste entre les rendus et déclenche un re-rendu quand il change
<code>useEffect</code>	exécuter du code après le rendu pour se synchroniser avec l'extérieur
<code>useContext</code>	lire une valeur partagée (fournie par un <code>Provider</code>) sans la faire descendre prop par prop (éviter le prop drilling)
<code>useRef</code>	conserver une valeur mutable entre les rendus (nœud DOM, id de timer, flag) sans provoquer de re-rendu
<code>useMemo</code> / <code>useCallback</code>	mémoriser une valeur calculée ou une fonction, pour éviter un recalcul ou une nouvelle référence à chaque rendu

🔗 useState — de plus près

```
const [count, setCount] = useState<number>(0);
```

- **Le state est un instantané.** Dans un rendu donné, `count` est une constante. Le setter ne le modifie pas : il demande un **nouveau rendu**.

```
setCount(count + 1);  
setCount(count + 1);           // ✗ +1 au total : count vaut toujours l'ancien  
setCount(c => c + 1);  
setCount(c => c + 1);           // ✔ +2 : forme fonctionnelle
```

- **Immuabilité obligatoire** : React compare les références.

```
posts.push(newPost); setPosts(posts); // ✗ même référence → aucun re-rendu  
setPosts(prev => [newPost, ...prev]); // ✔ nouveau tableau
```

🔗 useEffect — de plus près

```
useEffect(() => {  
  const id = setInterval(tick, 1000);  
  return () => clearInterval(id); // cleanup : au démontage et avant chaque ré-exécution  
}, [delay]); // dépendances : quand ré-exécuter ?
```

Deps	Ré-exécution
[]	une seule fois, au montage
[a, b]	à chaque changement de a ou b
absentes	à chaque rendu — presque toujours un bug

- S'exécute **après** que React a écrit dans le DOM

Piège fréquent

Un effet qui `setState` sur une dépendance qu'il modifie = boucle infinie.

Créer le projet

```
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
npm run dev
```

```
frontend/
├── index.html           ← le point d'entrée réel (pas dans public/)
├── vite.config.ts
├── tsconfig.json
└── src/
    ├── main.tsx        ← createRoot(...).render(<App />)
    └── App.tsx
```

```
npm run dev      # serveur de développement
npm run build    # tsc --noEmit puis build de production
npm run preview  # sert le build de prod en local
```

Vite, c'est quoi et pourquoi

Vite fait tourner votre projet en local (rechargement instantané à chaque sauvegarde) et prépare la version envoyée en production. Il remplace Create React App, aujourd'hui abandonné.

- Vite ne vérifie **pas** les types pendant que vous codez : seul `npm run build` le fait vraiment
- Prenez l'habitude de lancer un `build` avant de commit, pour ne pas découvrir une erreur de type trop tard

Attention

Les variables d'environnement préfixées `VITE_` finissent visibles dans le code envoyé au navigateur. N'y mettez jamais de secret (mot de passe, clé API).

Bloc 3 — React qui casse en production

Les vrais sujets

L'état d'écran : le pattern qui change tout

Avec 3 `useState` séparés, rien n'empêche des combinaisons absurdes : `loading=true` **et** `error="..."` **et** `posts=[]` en même temps. Le compilateur ne voit aucun problème, lui aussi les autorise.

✗ Avant

```
const [loading, setLoading] =
  useState(true);
const [error, setError] =
  useState<string | null>(null);
const [posts, setPosts] =
  useState<Post[]>([]);
```

✓ Après

```
type State<T> =
  | { status: "loading" }
  | { status: "error"; message: string }
  | { status: "empty" }
  | { status: "success"; data: T };
```

Pourquoi ce pattern ?

Un seul `status` possible à la fois : les états impossibles ne sont plus représentables. Le composant ne peut plus se retrouver dans deux états contradictoires en même temps.

Rendu exhaustif garanti par le compilateur

Le composant n'a plus qu'à lire `state.status` et choisir quoi afficher — un `switch` qui couvre les 4 cas possibles vus sur la slide précédente.

```
switch (state.status) {  
  case "loading": return <Spinner />;  
  case "error":   return <ErrorBox message={state.message} />;  
  case "empty":   return <EmptyState />;  
  case "success": return <PostList posts={state.data} />;  
  default: {  
    const _exhaustive: never = state;  
    return _exhaustive;  
  }  
}
```

Le `default` n'est pas là pour un cas réel : c'est un filet de sécurité. Si un `case` manque, `state` restant ne peut pas être `never`, et TypeScript refuse de compiler — au lieu de planter en prod.

useEffect : vous n'en avez probablement pas besoin

Ici, `count` ne fait que refléter `posts.length` : c'est une valeur **dérivée**, pas une donnée indépendante. Passer par un effet ajoute un rendu inutile (un premier rendu avec l'ancien `count`, puis un second déclenché par l'effet).

✗ Dériver un état avec un effet

```
const [posts, setPosts] =
  useState<Post[]>([]);
const [count, setCount] =
  useState(0);

useEffect(() => {
  setCount(posts.length);
}, [posts]); // rendu en trop
```

✓ Calculer pendant le rendu

```
const count = posts.length;
```

`useEffect` sert à **synchroniser avec l'extérieur** : réseau, DOM, abonnement, timer. Pas à réagir à un changement de props ou d'état.

useEffect + fetch = race condition

Un utilisateur ouvre le post 1, puis clique très vite vers le post 2. Deux requêtes partent, mais rien ne garantit l'ordre des **réponses** : si celle du post 1 arrive après celle du post 2, elle écrase l'écran avec les mauvaises données — alors que l'utilisateur regarde déjà le post 2.

✗ Sans annulation

```
useEffect(() => {  
  fetch(`/posts/${id}`)  
    .then(r => r.json())  
    .then(setPost);  
}, [id]);  
// la vieille réponse peut  
// écraser la nouvelle
```

✓ Avec AbortController

```
useEffect(() => {  
  const controller = new AbortController();  
  fetch(`/posts/${id}`, { signal: controller.signal })  
    .then(r => r.json())  
    .then(setPost)  
    .catch(e => {  
      if (e.name !== "AbortError") setError(e);  
    });  
  return () => controller.abort();  
}, [id]);
```

Le cleanup annule la requête du post précédent dès que `id` change : sa réponse, si elle arrive quand même, est ignorée au lieu d'écraser le bon état.

Erreurs classiques 1/2 — État et rendu

- **État dupliqué** : la même donnée dans deux `useState`, qui divergent
- **État global partout** : un Context pour ce qui pourrait être une prop
- `key={index}` dans une liste qui s'insère ou se supprime → composants mélangés, inputs qui gardent la mauvaise valeur. Utilisez l' `id` métier.
- `0 && <Badge />` qui affiche `0` à l'écran → préférez `count > 0 && ...`
- **Input non contrôlé devenu contrôlé** : `value={user.name}` alors que `user` est `undefined` au premier rendu

Erreurs classiques 2/2 — Typage et API

- **Pas de gestion du `catch`** : l'utilisateur voit un écran blanc
- **`!res.ok` non testé** : un 404 arrive dans `.then()` comme un succès
- **Token en clair dans le code** ou en dur dans un service

Question rapide

```
type State<T> =  
  | { status: "loading" }  
  | { status: "error"; message: string }  
  | { status: "empty" }  
  | { status: "success"; data: T };  
  
// Vous ajoutez { status: "unauthorized" } à l'union,  
// sans toucher au switch exhaustif qui gère l'affichage.
```

Que se passe-t-il ?

- A. Rien, le nouveau cas est ignoré silencieusement à l'exécution
- B. Erreur au build : `never` ne correspond plus au type restant
- C. Erreur seulement en production
- D. React affiche un warning dans la console

Exercice B — 15 min

Reprenez votre `apiGet` de l'exercice A. Le principe : sortir tout le `useEffect` dans un hook, pour que le composant ne fasse plus que du rendu.

Exercice — 15 minutes

1. Écrire un hook `usePosts()` qui retourne un `State<Post[]>`
2. Gérer les 4 états : loading, error, empty, success
3. Ajouter l'annulation au démontage
4. Le composant ne doit contenir **aucun** `useEffect`

Bloc 4 — Fullstack TypeScript

Une Promise, en 30 secondes

Une **Promise** représente une valeur **pas encore disponible**, mais qui le sera (ou échouera) plus tard — typiquement le résultat d'un appel réseau.

```
async function fetchPosts(): Promise<Post[]> {  
  const res = await fetch("/posts"); // attend la réponse HTTP  
  return res.json();                // attend le parsing du JSON  
}
```

- `Promise<Post[]>` se lit : "une valeur qui, une fois prête, sera un `Post[]` "
- `await` met la fonction en pause jusqu'à ce que la valeur soit prête, puis vous la donne directement — plus besoin de `.then(...)`

Ce type décrit **quand** la valeur arrive et de quel type elle sera censée être — mais rien ne garantit encore qu'elle le sera vraiment. C'est le sujet de la slide suivante.

Typage n'est pas validation

```
async function fetchPosts(): Promise<Post[]> {  
  const res = await fetch("/posts");  
  return res.json(); // ⚠ TypeScript vous croit sur parole  
}
```

Le raisonnement, étape par étape :

1. Vous annoncez `Promise<Post[]>` : vous **promettez** à TypeScript que la donnée sera un tableau de `Post`.
2. TypeScript vous fait confiance **sans jamais vérifier** : cette annotation n'existe qu'à la compilation, elle disparaît à l'exécution.
3. `res.json()` peut renvoyer littéralement n'importe quoi — un champ renommé, `null`, une page d'erreur HTML — et TypeScript ne voit rien de tout ça.

À retenir

Un type TypeScript est une promesse que le code se fait à lui-même. Il faut vérifier les données une seule fois, à la frontière — c'est le rôle de Zod, qui suit.

Zod — c'est quoi, et pourquoi

Zod est une librairie de validation à l'**exécution** (runtime) pour TypeScript.

Un `type` ou une `interface` s'efface à la compilation : à l'exécution, plus rien ne vérifie que les données reçues le respectent — exactement le problème de la slide précédente. Un schéma Zod, lui, existe encore quand le code tourne : il vérifie vraiment.

```
import { z } from "zod";

export const PostSchema = z.object({
  id: z.string(),
  content: z.string().min(1).max(500),
  author: z.object({ id: z.string(), username: z.string() }),
});

export type Post = z.infer<typeof PostSchema>; // le type se déduit du schéma
```

Zod : valider avec `safeParse`

Le schéma ne sert pas qu'à définir un type — on l'utilise pour vérifier une donnée reçue, ici la réponse de `fetchPosts` de tout à l'heure :

```
const result = PostSchema.safeParse(data);  
// result.success === true → result.data est un vrai Post, garanti  
// result.success === false → result.error décrit précisément ce qui cloche
```

À retenir

Une seule définition (le schéma) donne à la fois le type TypeScript et la vérification réelle. Jamais deux sources à synchroniser.

Auth – le flux

1. Saisie email / mot de passe
2. `POST /auth/login`
3. Vérification back (hash) + génération JWT
4. Stockage du token côté front
5. Envoi du token sur les routes protégées
6. Middleware back : vérifie le token **et l'ownership**
7. `401` ⇒ logout + redirection

Attention

Ownership : « ce post est-il bien le sien ? » se vérifie **côté back**. Cacher le bouton Supprimer n'est pas une sécurité.

Où stocker le token ?

	<code>localStorage</code>	Cookie <code>httpOnly</code>
Lisible en JS	oui	non
Vulnérable XSS	oui	non
Vulnérable CSRF	non	oui (mitigé par <code>SameSite</code>)
Simplicité	élevée	demande une config back

`localStorage` est acceptable pour ce projet, **si vous savez dire pourquoi**.

Architecture front : par couche ou par feature ?

✗ Par couche

```
src/  
├── components/  
├── services/  
├── hooks/  
├── types/  
└── pages/
```

Ajouter une feature touche 5 dossiers.

✓ Par feature

```
src/  
├── features/  
│   ├── posts/  
│   │   ├── PostList.tsx  
│   │   ├── usePosts.ts  
│   │   └── posts.api.ts  
│   └── auth/  
└── shared/  
    App.tsx
```

Ce qui change ensemble vit ensemble, et se supprime ensemble.

Bloc 5 — Tests

Vérifier que ça marche, pas seulement que ça compile

Quoi tester ?

Pas « quels outils », mais « quoi, concrètement ».

- **Les 4 états d'écran** (`State<T>` , Bloc 3) : loading, error, empty, success — 4 tests évidents, un par état
- **Une réponse API invalide** (schéma Zod, Bloc 4) : elle doit produire un état d'erreur, jamais un crash
- **L'ownership** : « ce post est-il bien le sien ? » — un test **côté back**, pas côté front

Tests — l'outillage

```
npm i -D vitest @testing-library/react @testing-library/user-event msw
```

- **Vitest** : le lanceur
- **Testing Library** : teste ce que l'utilisateur voit, pas l'implémentation
- **MSW** : intercepte le réseau, pas de mock de `fetch` à la main
- **Playwright** : un seul test bout en bout (login → poster → voir le post)

On ne teste pas `useState` . On teste **un comportement**.

Tests — un exemple

Un test qui vérifie qu'un formulaire vide affiche bien une erreur, sans jamais toucher au code du composant :

```
test("affiche une erreur si le contenu est vide", async () => {  
  render(<CreatePost />);  
  await userEvent.click(screen.getByRole("button", { name: /publier/i }));  
  expect(await screen.findByRole("alert")).toBeInTheDocument();  
});
```

- `render` affiche le composant, comme le ferait le navigateur
- `getByRole("button", ...)` trouve le bouton **comme un utilisateur le ferait** — jamais par `data-testid` ni par classe CSS
- `findByRole("alert")` attend que le message d'erreur apparaisse à l'écran

Requêtes par rôle et par label : si le test passe, l'accessibilité de base du composant l'est aussi.

Place au projet

En annexe de ce deck :

- Typage des props et children (`ReactNode`)
- Hooks avancés : `useRef` , `useMemo` , `useCallback` , `useContext`
- MSW : setup, handlers réseau et exercice de couverture des 4 états

Annexes — non traitées en cours

À consulter en autonomie

Typing props and children

```
type CardProps = {  
  title: string;  
  children: React.ReactNode; // n'importe quel contenu affichable  
};  
  
function Card({ title, children }: CardProps) {  
  return (  
    <section>  
      <h2>{title}</h2>  
      {children}  
    </section>  
  );  
}
```

- `ReactNode` : tout ce qui peut être rendu (élément, string, number, `null`, tableau)
- Les props sont typées comme n'importe quel objet — **c'est là que TypeScript vous sauve** : une prop oubliée ou mal nommée casse le build, pas la démo.

🔗 useRef, useMemo, useCallback, useContext

```
const inputRef = useRef<HTMLInputElement>(null);  
inputRef.current?.focus(); // accès direct à un nœud DOM
```

`useRef` : une boîte mutable qui **survit aux rendus** et ne déclenche **jamais** de re-rendu. Pour un id de timer, un flag, un nœud DOM. Pas pour de l'affichage.

```
const sorted = useMemo(() => [...posts].sort(byDate), [posts]);  
const onSelect = useCallback((id: string) => setSelected(id), []);
```

Optimisations. **Mesurez avant.** Un `useMemo` sur une addition coûte plus cher qu'il ne rapporte.

```
const { token, logout } = useAuth(); // custom hook autour de useContext
```

`useContext` : état partagé sans prop drilling. Pour l'auth, le thème. Pas pour tout.

Un **custom hook**, c'est juste une fonction qui commence par `use` et appelle d'autres hooks.

MSW — le setup

```
// src/test/server.ts
import { setupServer } from "msw/node";
import { http, HttpResponse } from "msw";

export const handlers = [
  http.get("/posts", () => HttpResponse.json([])),
];

export const server = setupServer(...handlers);
```

```
// src/test/setup.ts
import { server } from "../server";

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

```
// vite.config.ts
```

Exercice C — 20 min

Reprenez le hook `usePosts()` de l'exercice B et le composant qui l'utilise.

Exercice — 20 minutes

1. Écrire un handler MSW distinct pour chacun des 4 états (loading, error, empty, success)
2. Un test par état : requêtes par **rôle** et par **label**, jamais par `data-testid` ni par classe CSS
3. **Interdit** : tester l'état interne du hook — aucun appel direct à `usePosts` hors du composant